TITRE PROFESSIONNEL DWWM

Dossier de projet

L'Aiguille des Volcans



Application Symfony et base de données MariaDB

Aureo Ruffier

Centre de formation : Institut de la PME
Formation du 4 mai au 14 août 2026
Soutenance du 29 octobre 2026

Titre professionnel Développeur web et web mobile
Réalisation individuelle

Sommaire

1 Présentation du projet 4
1.1 Mon parcours et le contexte 4
1.2 Le cahier des charges 5
1.3 Les usages du site 6
2 Conception du site 7
2.1 Ma méthode de travail 7
2.2 Acteurs et cas d'utilisation 8
2.3 Aller de la découverte à la demande 9
2.4 Les couleurs et la présentation 10
2.5 Préparer les écrans sur ordinateur 11
2.6 Adapter les mêmes contenus 12
3 Outils et environnement 13
3.1 Les technologies choisies 13
3.2 Comprendre les services Docker 14
3.3 Git et la préparation du front-end 15
4 Développement du back-end 16
4.1 Les entités et leurs relations 16
4.2 Modèle relationnel et structure de la base 17
4.3 Créer et faire évoluer la base 18
4.4 Le trajet d'une requête 19
4.5 Filtrer les produits d'une catégorie 20
4.6 Collecter les informations utiles 22
4.7 Valider puis enregistrer le devis 23
4.8 Consulter et suivre un devis 25
4.9 Créer lire modifier et supprimer 26
4.10 Authentifier les utilisateurs 27
4.11 Modération des avis 28

5 Développement du front-end 29
5.1 Construire les pages avec Twig 29
5.2 Présenter les réalisations 30
5.3 Rendre les interfaces utilisables 31
5.4 Ajouter des comportements dans le navigateur 32
6 Sécurité et confidentialité 34
6.1 Les protections appliquées 34
6.2 Protéger les fichiers de devis 35
6.3 Informer les utilisateurs 36
7 Vérification du site 37
7.1 Les tests et leurs résultats 37
7.2 Difficultés rencontrées et solutions apportées 39
8 Publication du site 40
8.1 La première mise en ligne sur OVH 40
9 Bilan du projet 41
9.1 Le résultat et les prochaines étapes 41
Annexe A Maquette du devis sur ordinateur 42
Annexe B Maquette du devis sur mobile 43
Annexe C Sources et repères techniques 44
Annexe D Exemple SQL et jeu d'essai 45

1 Présentation du projet

1.1 Mon parcours et le contexte

J'ai réalisé L'Aiguille des Volcans dans le cadre de ma formation au titre professionnel Développeur web et web mobile à L'Institut de la PME, du 4 mai au 14 août 2026. J'ai commencé à apprendre la programmation au début de l'année 2026. Ce projet m'a permis de mettre en pratique les connaissances du cours dans une application complète, depuis la préparation des écrans jusqu'à la publication d'une première version.

L'application présente une activité de broderie et de personnalisation textile portée par Oceane Roux, à Volvic. Le besoin est concret : montrer le savoir-faire de l'atelier et permettre à une personne intéressée de transmettre les informations nécessaires pour préparer un devis. Une réalisation textile dépend du support, du marquage, de la quantité et du motif. Le site doit donc guider le visiteur avant le premier échange avec l'atelier.

J'ai développé le projet seul. Pour avancer, je me suis appuyé sur les cours, les échanges avec les enseignants et les collègues de formation, ainsi que sur la documentation des outils utilisés. J'ai appris à relier les différentes parties du site et à vérifier le résultat dans le navigateur et avec des tests.

Ce dossier présente les besoins, les maquettes, la base de données, les principales fonctionnalités et les vérifications de l'application. Les captures montrent la version locale actuelle. Les enregistrements utilisés pour illustrer les comptes, les demandes et les avis sont fictifs. La soutenance est prévue le 29 octobre 2026.



Figure 1 Accueil de L'Aiguille des Volcans dans la version locale

1.2 Le cahier des charges

Le site s'adresse aux particuliers, aux associations et aux professionnels qui souhaitent personnaliser un textile. Un visiteur doit comprendre les prestations disponibles, voir des exemples de réalisations et trouver rapidement le formulaire de devis. L'atelier doit pouvoir mettre à jour ses contenus et suivre les demandes reçues depuis un espace réservé.

J'ai retenu une première version centrée sur la présentation de l'activité, le catalogue et la demande de devis. Les produits sont des exemples avec un prix indicatif éventuel. Le prix définitif dépend de la demande et des échanges avec l'atelier. Cette distinction explique le choix d'un formulaire de devis plutôt qu'un parcours de commande avec paiement.

Besoin	Réponse apportée dans le site
Présenter l'atelier	Accueil et pages de présentation des services et du savoir-faire
Montrer les réalisations	Galerie, catégories et fiches des produits actifs
Préparer un devis	Formulaire avec coordonnées, support, marquage, quantité et logo éventuel
Recueillir les retours	Avis associés à un compte et publiés après modération
Mettre à jour le contenu	Administration des produits, catégories, avis et utilisateurs
Suivre les demandes	Liste des devis et changement de statut par l'administratrice

J'ai séparé les contenus éditoriaux, qui présentent l'atelier, des données qui évoluent régulièrement. Les produits, catégories, utilisateurs, avis et demandes de devis sont enregistrés dans MariaDB. L'administratrice peut modifier ces éléments sans intervenir dans les fichiers du site.

Le périmètre reste adapté à mon niveau de formation et aux besoins de l'activité. Il comprend un site utilisable sur ordinateur et téléphone, une base relationnelle et un espace d'administration. La mise en place effective des notifications par courriel dépend encore de la création et de la configuration de la boîte de messagerie.

1.3 Les usages du site

J'ai distingué trois situations d'utilisation. Le visiteur consulte librement les pages publiques et peut demander un devis sans créer de compte. L'utilisateur connecté dispose en plus de la possibilité de déposer un avis. L'administratrice accède aux écrans de gestion après authentification avec le rôle prévu pour cet espace.

Cette séparation m'a aidé à décider quelles informations afficher et quelles actions protéger. Elle évite, par exemple, d'imposer une inscription pour une simple demande commerciale, tout en rattachant chaque avis à un compte.

Compétence mise en pratique	Exemple dans le projet
Installer un environnement de travail	Docker, PHP, MariaDB, Node et gestion des dépendances
Maquetter les interfaces	Wireframes et maquettes Figma sur ordinateur et mobile
Réaliser les interfaces statiques	Structure HTML, Twig, Bootstrap et CSS personnalisé
Développer les interactions des interfaces	(JS) Thème visuel, agrandissement des images et aperçu de fichier
Mettre en place une base relationnelle	Entités, relations, migrations et base MariaDB
Développer les accès aux données	Repositories Doctrine et requêtes paramétrées
Développer les composants métier	Traitement des devis, gestion du catalogue et modération des avis
Documenter le déploiement	Transfert FileZilla et export et import SQL dans phpMyAdmin

Dans les chapitres suivants, j'explique ces compétences à partir de fonctionnalités visibles. La demande de devis sert de fil conducteur : elle relie une interface, un formulaire validé, une entité, l'enregistrement en base, un fichier éventuel et le suivi administratif.

Les mesures de protection sont présentées avec le fonctionnement concerné. Je décris également les tests et les choix de confidentialité, car une application qui collecte des coordonnées doit tenir compte de leur utilisation et de leur accès.

2 Conception du site

2.1 Ma méthode de travail

J'ai organisé le développement autour des besoins du site et des fonctionnalités à construire. La démarche a consisté à préciser les informations à présenter, préparer les écrans, définir les données puis développer les pages et leurs traitements. Les maquettes et la structure de données m'ont servi de repères, avec des ajustements lorsque l'intégration faisait apparaître un besoin supplémentaire.

Pour le catalogue, j'ai d'abord identifié les informations d'un produit et ses catégories. J'ai ensuite relié la lecture de ces données à la galerie et à la fiche détaillée. Pour les devis, j'ai travaillé sur les champs utiles au visiteur, leur validation puis leur affichage dans l'administration. Cela m'a permis d'avancer par parcours complet et de voir le résultat de chaque étape.

Étape de conception et de réalisation	Résultat recherché
Comprendre le besoin	Définir les pages et les actions utiles à l'atelier et au visiteur
Préparer les écrans	Fixer la navigation, la présentation et la disposition des formulaires
Définir les données	Identifier les entités et leurs relations
Développer les parcours	Relier les pages, les formulaires et la base de données
Vérifier et corriger	Contrôler les affichages, les erreurs et les droits d'accès
Publier une première version	Installer les fichiers et la base sur l'hébergement OVH

Le projet est conservé sur GitHub. J'ai créé des issues pour identifier des tâches et des milestones pour envisager leur regroupement par étapes. Je n'ai pas maintenu leur suivi pendant tout le développement. En revanche, j'ai continué à faire des commits et des push régulièrement afin de conserver sur GitHub une version actualisée du projet.

Cette expérience m'a montré que la sauvegarde du code et le suivi des tâches sont deux habitudes différentes. La prochaine amélioration de mon organisation sera de tenir les issues à jour à mesure que les fonctionnalités sont terminées. Le dépôt reste déjà un support pour retrouver les changements du code.

2.2 Acteurs et cas d'utilisation

Le diagramme de cas d'utilisation présente les fonctionnalités du point de vue des personnes qui utilisent le site. Les ovales décrivent des objectifs, comme consulter une réalisation ou transmettre une demande. Ils ne représentent pas les tables de la base de données.

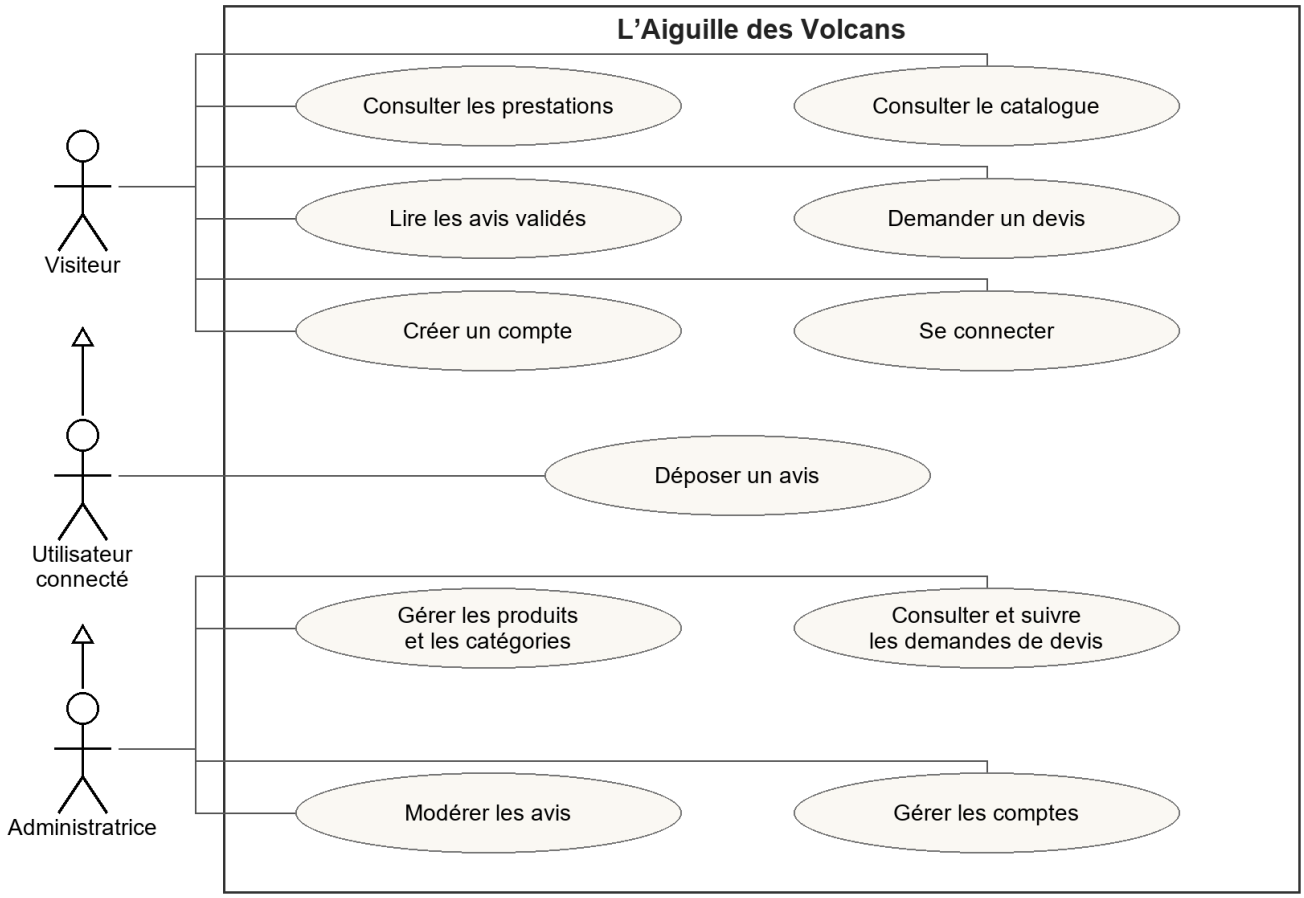


Figure 2 Cas d'utilisation de la version présentée

Un utilisateur connecté conserve les possibilités du visiteur et peut déposer un avis. L'administratrice dispose de ces usages ainsi que de fonctions de gestion. La consultation du site public ne donne aucun droit de modification des données.

Ce diagramme m'aide à expliquer le périmètre fonctionnel avant de présenter le code. Le traitement d'un devis commence par une action du visiteur et se poursuit dans un espace accessible uniquement à l'administratrice.

2.3 Aller de la découverte à la demande

J'ai conçu la navigation pour que le visiteur trouve les trois informations essentielles : ce que propose l'atelier, des exemples de réalisations et le moyen de demander un devis. Le menu principal reste disponible sur les pages publiques. Sur un écran étroit, il est regroupé dans un menu adapté à la largeur disponible.

Le parcours principal commence sur l'accueil. Le visiteur peut découvrir les services, ouvrir la galerie et consulter un produit. Il peut ensuite accéder à la demande de devis. Le formulaire rassemble les éléments qui permettront à l'atelier de comprendre le besoin et de reprendre contact.

Un second parcours concerne les avis. La lecture des avis publiés permet de découvrir des retours sur les réalisations. Le dépôt d'un avis demande une connexion. Après l'envoi, l'avis reste en attente de modération, ce qui sépare le témoignage de sa publication.

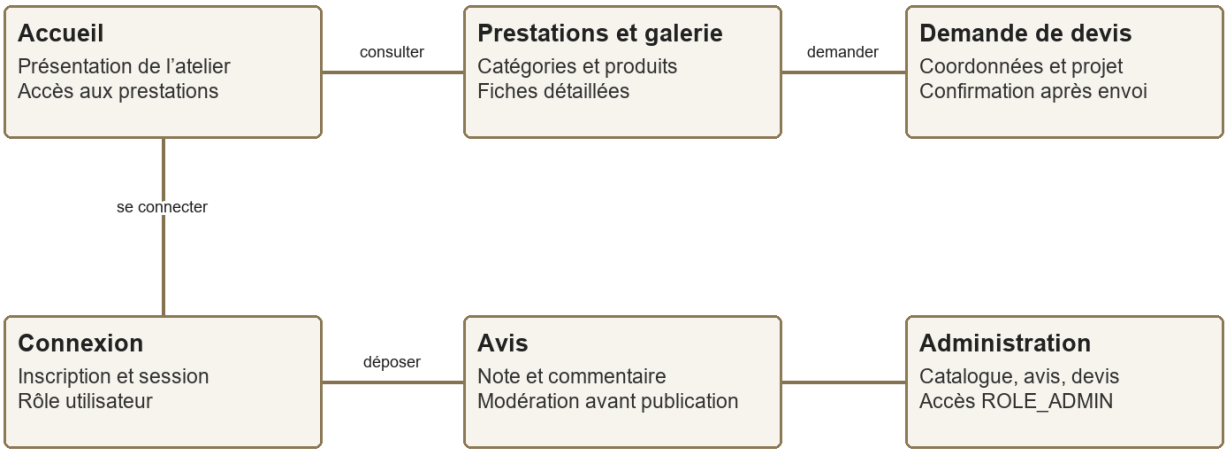


Figure 3 Organisation générale des pages et des parcours

Le parcours de l'administratrice est différent : connexion, tableau de bord, choix d'une rubrique puis consultation ou modification d'un enregistrement. Par exemple, une demande reçue peut être ouverte dans la liste des devis et son statut mis à jour.

Je conserve des intitulés simples dans les menus et les boutons. Le visiteur doit savoir ce qu'un clic va provoquer. La navigation et les messages de confirmation participent à la compréhension du site autant que son apparence.

2.4 Les couleurs et la présentation

La conception graphique doit refléter une activité artisanale et donner une place importante aux photographies des textiles. J'ai utilisé une palette claire avec des teintes chaudes et des accents dorés. Les couleurs servent à distinguer les boutons et certains éléments de présentation, tout en gardant le contenu lisible.

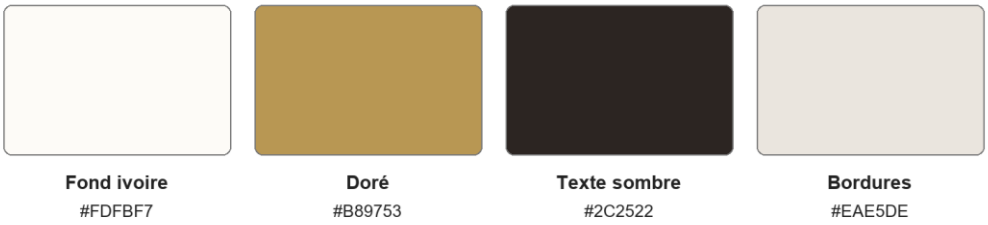


Figure 4 Principales couleurs de la présentation claire

Le fond clair évite de concurrencer les photos des réalisations. Les tons sombres sont utilisés pour les textes. La couleur dorée revient sur les boutons et les détails visuels, ce qui permet d'identifier les actions importantes. Les titres, les espacements et les cartes suivent une présentation commune entre les pages.

Le logo et les images de l'atelier donnent une identité au site. Les photographies ont un rôle fonctionnel : elles montrent le rendu d'une broderie et aident le visiteur à se projeter dans sa propre demande. Je les accompagne de textes pour expliquer les prestations et les caractéristiques des produits.

Le site propose également un thème sombre. Le choix est appliqué à l'interface et mémorisé dans le navigateur. Cette option a été l'occasion de travailler sur les variables de couleurs et sur une interaction JavaScript simple, tout en conservant les mêmes contenus.

Les maquettes définissent l'intention de départ. Pendant l'intégration, j'ai adapté les tailles, les espacements et certains regroupements pour obtenir un résultat cohérent avec Bootstrap et avec les contenus réellement disponibles.

Les textes courants utilisent une police à empattements, avec Georgia et une police de remplacement. La navigation utilise une police système sans empattements. Cette différence distingue les textes de présentation des commandes de l'interface. Les variables CSS regroupent les couleurs afin de les réutiliser et de les adapter au thème sombre.

Extrait du dépôt • assets/styles/page/page.css • lignes 2 à 10

```
 2  :root {
 3      --bg-color: #fdfbf7;
 4      --primary-color: #b89753;
 5      --text-dark: #2c2522;
 6      --card-bg: #ffffff;
 7      --surface-color: #f8f9fa;
 8      --border-color: #eae5de;
 9      --muted-text: #555;
10  }
```

2.5 Préparer les écrans sur ordinateur

Avant de détailler l'apparence, les wireframes permettent de placer les grandes zones d'une page : menu, titre, contenu et boutons. Ils aident à vérifier l'ordre de lecture sans passer immédiatement du temps sur les couleurs et les images.

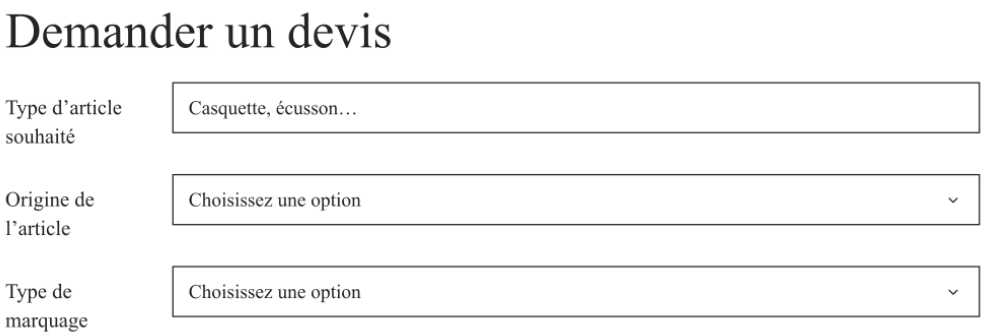


Figure 5 Extrait du wireframe du formulaire de devis

J'ai ensuite utilisé les maquettes Figma pour préciser la présentation. Le formulaire de devis est un bon exemple, car il contient plusieurs groupes d'informations. Les coordonnées doivent être faciles à repérer, puis les caractéristiques du textile et du marquage doivent suivre une logique compréhensible.

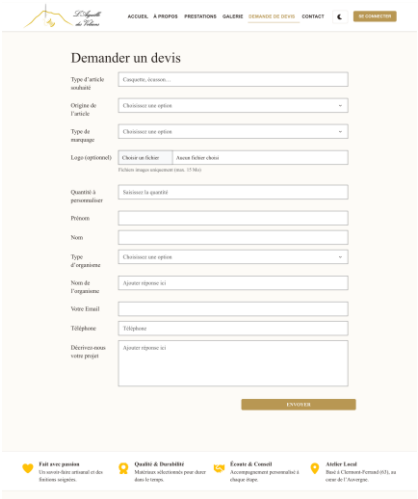


Figure 6 Maquette du devis sur ordinateur issue des exports du projet

Ces supports m'ont permis de préparer l'intégration et de comparer le résultat obtenu à l'écran avec le projet graphique. Ils sont complétés par les captures de l'application présentées dans les chapitres de développement.

La maquette complète est reprise en annexe pour permettre une lecture plus précise de sa disposition. La conception et l'intégration restent liées : la maquette guide le travail, puis le comportement réel du formulaire permet de vérifier que cette disposition est pratique.

2.6 Adapter les mêmes contenus

Le visiteur peut découvrir l'atelier depuis un téléphone. Il doit retrouver les mêmes informations et pouvoir demander un devis sans utiliser un ordinateur. J'ai donc prévu une disposition plus étroite des écrans et des champs du formulaire.

Sur ordinateur, certaines informations peuvent être placées côte à côte. Sur mobile, elles sont regroupées dans une lecture verticale. Les boutons doivent rester identifiables et les textes ne doivent pas provoquer de défilement horizontal. Les photographies conservent leur rôle, mais leur taille s'adapte à la place disponible.



Figure 7 Extrait de la maquette mobile du devis

Pendant l'intégration, j'ai comparé le rendu à différentes largeurs. Cette vérification concerne le menu, les cartes de produits et les formulaires. Une interface responsive est une interface dont la disposition s'adapte à l'écran ; elle ne consiste pas seulement à réduire toute la page.

La maquette mobile complète figure en annexe. Dans le chapitre consacré au front-end, une capture montre également le résultat réel dans un navigateur configuré avec une largeur de téléphone.

3 Outils et environnement

3.1 Les technologies choisies

J'ai développé le site avec Symfony, un framework PHP. Un framework fournit une organisation et des outils pour construire une application. Symfony m'a permis de travailler avec des routes, des contrôleurs, des formulaires et une gestion des comptes dans un même projet.

Outil	Rôle dans L'Aiguille des Volcans
PHP et Symfony 7.4	Exécuter les traitements sur le serveur et organiser l'application
Twig	Créer les pages HTML à partir de modèles et de données
Doctrine ORM	Relier les objets PHP aux tables de la base
MariaDB	Conserver les données dans une base relationnelle SQL
Bootstrap 5.3	Aider à construire la grille et les composants responsive
CSS personnalisé	Adapter l'apparence à l'identité de l'atelier
JavaScript	Ajouter les interactions dans le navigateur
Webpack Encore et Node	Préparer les fichiers CSS et JavaScript destinés au navigateur
Docker	Faire fonctionner les services nécessaires en développement
Git et GitHub	Conserver les versions du code et les transférer vers le dépôt distant

Ces choix sont cohérents avec les technologies travaillées pendant la formation. Je peux expliquer le rôle de chaque outil à partir d'une page du site : Symfony reçoit la demande, Doctrine récupère les produits et Twig construit leur affichage. Bootstrap et le CSS définissent la disposition visible.

Je n'ai pas ajouté un framework JavaScript complet pour les interactions du site. Les besoins de cette version, comme un aperçu d'image ou le changement de thème, peuvent être réalisés avec quelques scripts dédiés. Cela me permet de garder une organisation adaptée au projet et à mon apprentissage.

3.2 Comprendre les services Docker

Le projet est développé sur Windows avec les fichiers dans Ubuntu sous WSL. Docker permet de lancer les services nécessaires sans installer chaque composant directement dans le même système. Chaque service a une fonction précise et communique avec les autres.

Le service PHP exécute l'application Symfony. Le serveur web Caddy reçoit les demandes du navigateur. MariaDB conserve les données et phpMyAdmin permet de les consulter avec une interface. Le service Node prépare les ressources du front-end.

Service local	Utilisation pendant le développement
PHP	Commandes Symfony, application, migrations et tests
Caddy	Accès au site dans le navigateur
MariaDB	Tables et enregistrements du projet
phpMyAdmin	Consultation de la structure et export des données
Node	Installation et compilation des ressources du navigateur

J'ai dû apprendre à distinguer une erreur de l'application d'un problème d'environnement. Par exemple, si la base n'est pas disponible, le traitement ne peut pas lire les produits même si le contrôleur est correct. Vérifier les services constitue donc une première étape avant de chercher une erreur dans le code.

Les dépendances PHP sont décrites dans composer.json et les dépendances du front-end dans package.json. Les fichiers de verrouillage permettent de retrouver les versions utilisées. Les paramètres propres à l'environnement, comme l'accès à la base et les secrets, sont configurés séparément du code destiné au dépôt.

Cet environnement m'a donné un cadre pour développer, charger les données de démonstration et exécuter les vérifications avant d'envoyer une mise à jour sur l'hébergement.

3.3 Git et la préparation du front-end

Git conserve l'historique du code. Un commit correspond à un ensemble de changements enregistré avec un message. Le push envoie ces commits sur GitHub. Pendant le développement, j'ai régulièrement utilisé ce fonctionnement pour maintenir le dépôt distant à jour.

Le dépôt est disponible à l'adresse github.com/aureopires/projet_broderie. Il contient les fichiers de l'application et son historique. Les issues et milestones ont été créées comme outils de préparation des tâches, mais leur suivi n'a pas été continu. Dans ce dossier, je distingue cette préparation de l'usage régulier de Git pour conserver le travail.

Lorsque je modifie une fonctionnalité, je peux retrouver les fichiers concernés et comparer leur nouvelle version à la précédente. Les messages de commit aident à comprendre l'objectif d'une modification. Le dernier ensemble de corrections étudié pour ce dossier concerne notamment la validation des formulaires et la confidentialité des données.

La préparation des ressources du navigateur est une autre étape du travail. Les fichiers de départ se trouvent dans assets. Webpack Encore prépare les fichiers qui seront utilisés depuis public/build. Le navigateur doit charger ces fichiers compilés, et non les outils Node nécessaires à leur préparation.

Fichier ou dossier	Ce qu'il contient
src	Classes PHP de l'application
templates	Modèles Twig des pages
assets	CSS, JavaScript et ressources de départ
migrations	Évolutions de la structure de la base
tests	Vérifications automatiques
public	Point d'entrée web et ressources accessibles au navigateur

Cette séparation m'aide à retrouver les fichiers à modifier. Pour changer la mise en page, je commence par Twig et les styles. Pour changer les données présentées, je remonte vers le contrôleur et le repository.

Une mise à jour suit un cycle simple : modifier les fichiers, vérifier le résultat, créer un commit puis envoyer les changements avec un push. La publication sur OVH reste une opération distincte du push GitHub. Le dépôt actualisé ne modifie donc pas automatiquement le site hébergé.

4 Développement du back-end

4.1 Les entités et leurs relations

J'ai défini cinq entités métier : User, Product, Category, Review et QuoteRequest. Une entité représente un type d'information utilisé par l'application. Ses propriétés deviennent des champs de la base, et ses relations représentent les liens entre les enregistrements.

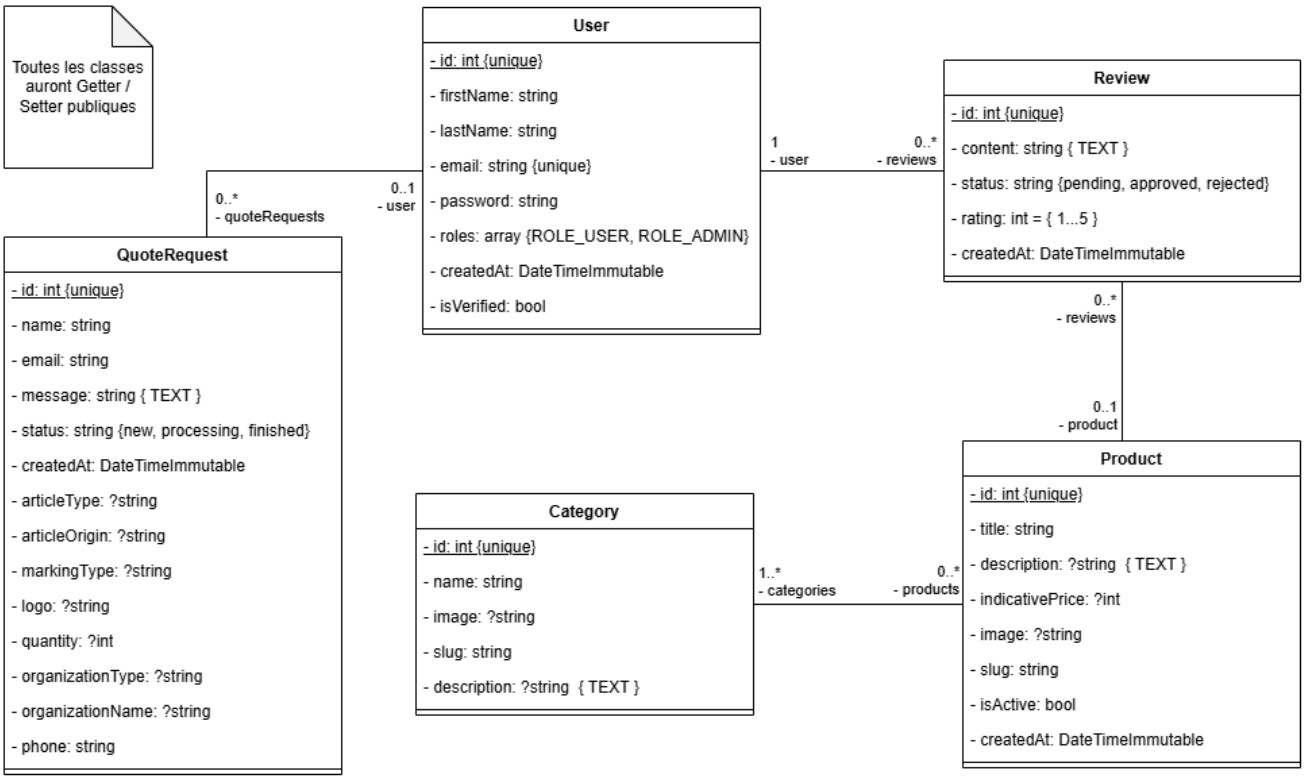


Figure 8 Diagramme de classes fourni pour le projet

Un produit peut appartenir à plusieurs catégories. Une catégorie peut regrouper plusieurs produits. Cette relation est appelée plusieurs à plusieurs. Un utilisateur peut transmettre plusieurs demandes de devis et déposer plusieurs avis. Une demande peut aussi venir d'un visiteur non connecté ; son lien vers User reste alors facultatif.

Un avis appartient à un utilisateur. Il peut concerner un produit, mais cette association est facultative. Les champs précédés d'un point d'interrogation dans le diagramme représentent des valeurs facultatives dans le modèle. La validation du formulaire peut imposer un renseignement dans un parcours précis sans rendre obligatoire chaque colonne de la base.

Cette distinction m'a aidé à comprendre la différence entre la structure des données et les règles d'une interface. Elle permet de conserver des informations historiques ou des données propres à une situation tout en contrôlant les nouvelles saisies.

4.2 Modèle relationnel et structure de la base

La base MariaDB traduit ces entités en tables. Une clé primaire identifie chaque ligne. Une clé étrangère relie une ligne à une autre table. La table product_category fait le lien entre les produits et leurs catégories, puisqu'un seul champ dans Product ne suffirait pas pour plusieurs catégories.

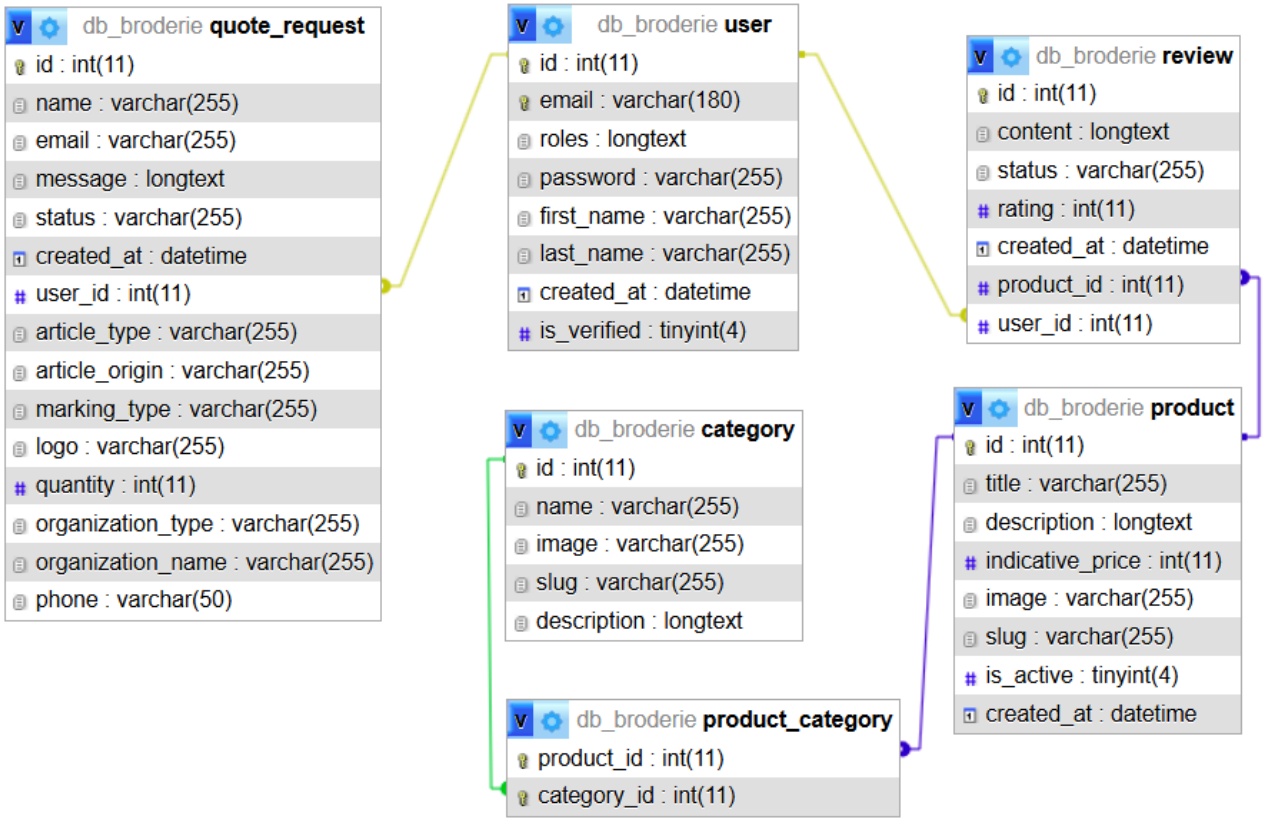


Figure 9 Structure physique présentée dans phpMyAdmin

Par exemple, review.user_id correspond à l'utilisateur qui a déposé l'avis. quote_request.user_id permet de rattacher une demande à un compte lorsqu'il existe. Les coordonnées saisies dans le devis restent dans quote_request afin que l'atelier dispose des informations de contact de cette demande.

J'ai utilisé des types adaptés : texte pour les descriptions et les messages, entier pour la quantité et la note, date pour la création, booléen pour l'activation d'un produit. L'adresse électronique du compte est unique afin d'éviter deux comptes avec la même adresse.

Le schéma facilite la compréhension des liens et complète le diagramme de classes. Le contrôle de Doctrine a confirmé que le mappage des entités est valide et que la structure de la base locale correspond à la version actuelle du code.

4.3 Créer et faire évoluer la base

J'ai utilisé Doctrine pour générer la structure de la base locale à partir des entités. Une migration décrit une évolution de cette structure sous forme de commandes SQL. Les migrations sont conservées dans le dépôt pour expliquer les changements de tables et permettre de les appliquer dans un ordre défini.

Le passage d'une relation entre Product et Category à une relation plusieurs à plusieurs illustre l'intérêt de cette méthode. La base doit contenir une table de liaison et des clés correspondant aux deux entités. Modifier uniquement l'écran ne suffirait pas : le modèle, la structure et le traitement doivent rester cohérents.

Pour préparer les démonstrations, le projet dispose de fixtures. Ce sont des données de départ chargées pour disposer de produits, catégories, comptes, avis et devis sans tout saisir à la main. Faker permet de produire des valeurs fictives. Ces enregistrements facilitent les essais des listes et des relations.

Pour la première publication, j'ai exporté la base locale puis importé le fichier SQL dans phpMyAdmin sur l'hébergement. Avant une évolution d'une base déjà utilisée, je prévois une sauvegarde et je vérifie les changements de structure. Une nouvelle importation complète ne doit pas effacer les demandes reçues entre deux versions.

Pour préparer un environnement local, les principales étapes sont l'installation des dépendances, la configuration de la connexion, la création de la base puis l'application des migrations. Les commandes Symfony permettent de décrire ces actions et d'en contrôler le résultat.

```
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate
php bin/console doctrine:schema:validate
```

Ces commandes sont exécutées dans l'environnement PHP du projet. Le chargement de fixtures est réservé à une base de démonstration, car il peut remplacer son contenu.

4.4 Le trajet d'une requête

Symfony organise l'application en composants. Pour expliquer leur rôle, je prends l'exemple d'un visiteur qui ouvre la galerie. Le navigateur envoie une requête à une adresse. La route correspondante indique le contrôleur à exécuter.

Le contrôleur demande les données utiles au repository. Celui-ci prépare la recherche avec Doctrine, qui communique avec MariaDB. Le contrôleur transmet ensuite les produits à un modèle Twig. Twig construit une page HTML qui est renvoyée au navigateur.

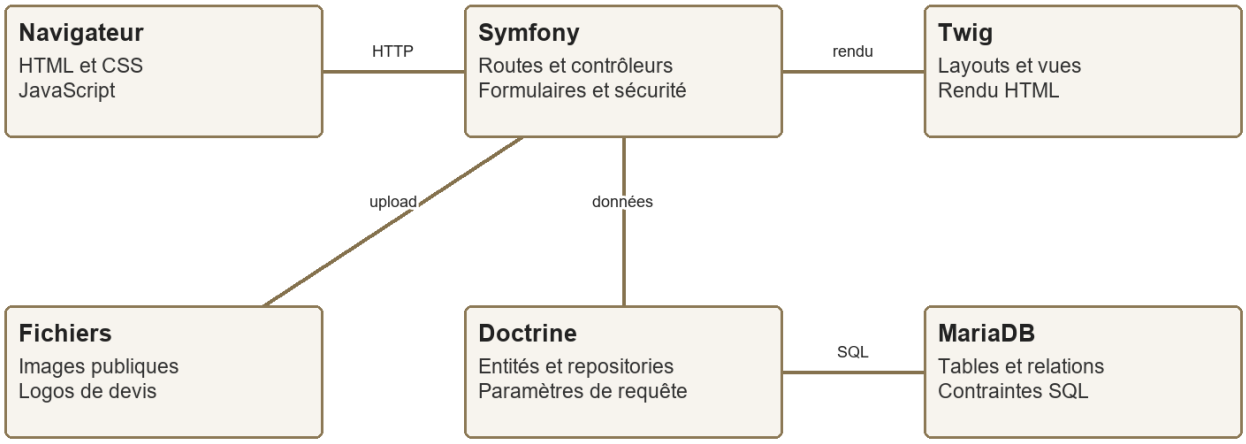


Figure 10 Organisation de l'application et accès à la base

Cette organisation suit le principe MVC, pour modèle, vue et contrôleur. Le modèle concerne les données et leurs traitements, la vue concerne leur présentation, et le contrôleur coordonne la réponse à une demande. Dans mon projet, les entités et repositories participent au modèle, les fichiers Twig aux vues et les contrôleurs aux parcours du site.

La séparation m'aide à corriger un problème. Si le produit existe dans la base mais n'apparaît pas, je vérifie la requête et les valeurs envoyées au template. Si les données sont bonnes mais la carte est mal présentée, je vérifie Twig et le CSS.

Symfony fournit les services nécessaires au contrôleur, par exemple le repository ou le gestionnaire des entités. Cette injection de dépendances évite de recréer la connexion à la base dans chaque action. Je peux concentrer l'action sur son objectif : lire, afficher ou enregistrer les informations.

4.5 Filtrer les produits d’une catégorie

Le repository rassemble les recherches liées à une entité. ProductRepository contient notamment une méthode pour récupérer les produits actifs d’une catégorie. Cela évite de recopier la même recherche dans plusieurs contrôleurs.

Une même recherche avec Doctrine et en SQL

Extrait du dépôt • src/Repository/ProductRepository.php • méthode findByCategorySlug

QueryBuilder Doctrine	SQL de lecture correspondant

`return $this->createQueryBuilder('p')`
`->innerJoin('p.categories', 'c')`
`->andWhere('p.isActive = :active')`
`->andWhere('c.slug = :slug')`
`->setParameter('active', true)`
`->setParameter('slug', $slug)`
`->orderBy('p.createdAt', 'DESC')`
`->getQuery()`
`->getResult();`
	`SELECT p.*`
	`FROM product AS p`
	`INNER JOIN product_category AS pc`
	`ON pc.product_id = p.id`
	`INNER JOIN category AS c`
	`ON c.id = pc.category_id`
	`WHERE p.is_active = :active`
	`AND c.slug = :slug`
	`ORDER BY p.created_at DESC;`

À gauche, le code utilise les propriétés des entités. À droite, la traduction pédagogique rend explicites les deux jointures SQL de la relation plusieurs à plusieurs. Les paramètres correspondent à active = true et au slug demandé. Ce SQL illustre la recherche ; il ne s’agit pas d’une capture de la requête générée par Doctrine.

La lettre p représente Product dans la requête et c représente ses catégories. innerJoin relie les deux objets grâce à leur association Doctrine. La recherche conserve les produits actifs et la catégorie dont le slug correspond à l’adresse demandée. Les résultats sont triés par date de création décroissante.

Le slug est un identifiant lisible utilisé dans les adresses, par exemple pour retrouver une catégorie sans afficher seulement son numéro. Sa valeur est transmise avec setParameter. Elle reste séparée de l’expression de la requête, ce qui participe à la prévention des injections SQL.

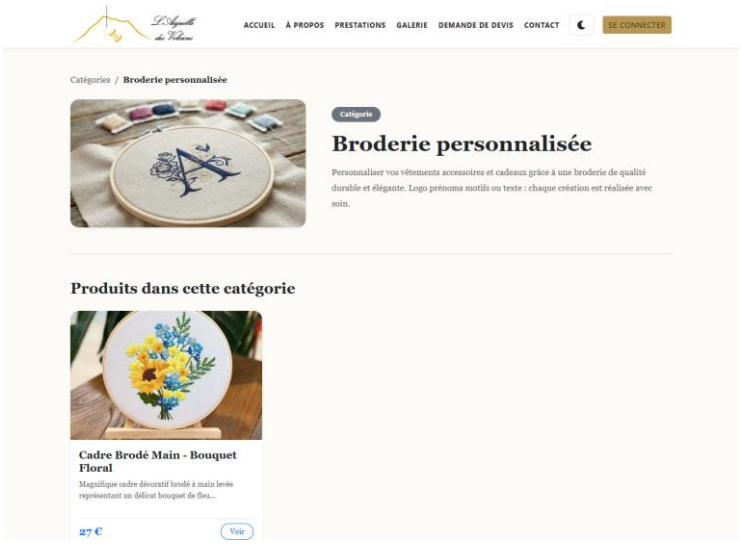


Figure 11 Page des réalisations d'une catégorie

J'ai vérifié que les différents accès à une catégorie appliquent la même règle d'activation. Un produit désactivé ne doit pas réapparaître parce que le visiteur utilise une autre adresse du catalogue. Ce cas est couvert par les tests du repository et des pages concernées.

La requête fournit les données ; le template garde la responsabilité de construire les cartes. Cette limite entre accès aux données et affichage est une des notions que j'ai approfondies pendant le projet.

4.6 Collecter les informations utiles

La demande de devis est une fonctionnalité centrale. Elle doit permettre à une personne de décrire son besoin, puis à l'atelier de comprendre la demande sans chercher les informations dans plusieurs messages.

Le formulaire recueille les coordonnées et les caractéristiques de la demande : type d'article, origine du textile, technique de marquage, quantité, situation du demandeur et message. Un logo peut être ajouté si le projet en comporte un. Les informations concernant une organisation sont adaptées à cette situation et ne décrivent pas nécessairement chaque particulier.

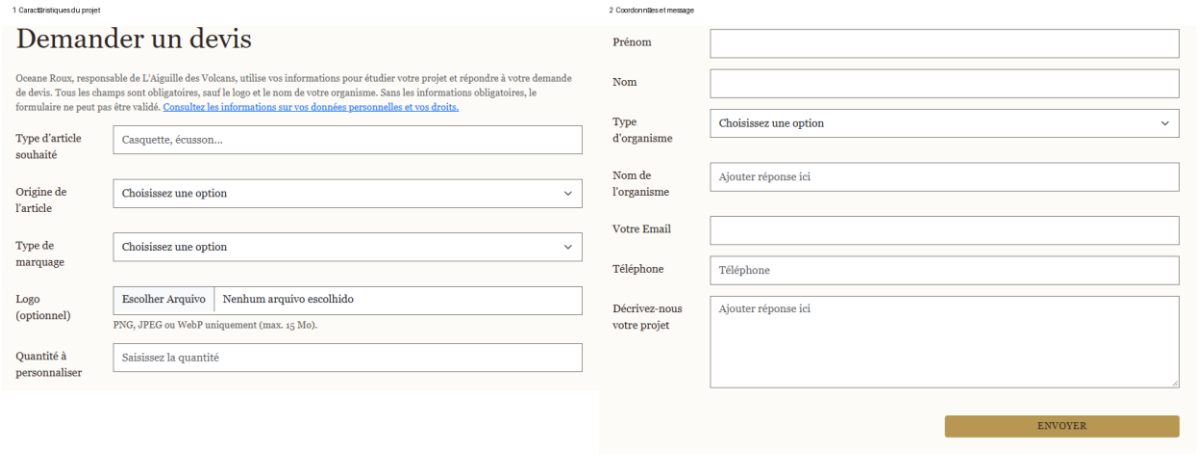


Figure 12 Deux extraits du formulaire actuel de devis

J'ai utilisé un type de formulaire Symfony pour déclarer les champs, les choix proposés et les contraintes. Le formulaire associe les données à QuoteRequest. Certains champs sont traités séparément : le fichier envoyé n'est pas directement un contenu à enregistrer dans une colonne de l'entité.

Une partie de la vérification est visible dans le navigateur, mais la validation du serveur reste indispensable. Une quantité positive, une adresse de courriel correcte et les champs obligatoires sont contrôlés avant l'enregistrement. Si la saisie n'est pas valable, le visiteur retrouve le formulaire avec des erreurs pour corriger sa demande.

L'attribut min aide la saisie dans le navigateur. NotBlank et GreaterThanOrEqual vérifient réellement la quantité sur le serveur. L'extrait suivant montre cette différence.

Extrait du dépôt • src/Form/QuoteRequestType.php • lignes 52 à 60

```
52  ->add('quantity', IntegerType::class, [
53      'label' => 'Quantité à personnaliser',
54      'required' => true,
55      'attr' => ['placeholder' => 'Saisissez la quantité', 'min' => 1],
56      'constraints' => [
57          new NotBlank(message: 'Veuillez indiquer une quantité.'),
58          new GreaterThanOrEqual(1),
59      ],
60  ])
```

4.7 Valider puis enregistrer le devis

Le contrôleur du devis crée une nouvelle demande et construit le formulaire. Après l'envoi, Symfony lit les valeurs reçues et applique les contraintes. Le contrôleur poursuit le traitement lorsque le formulaire est envoyé et valide.

Il prépare alors les informations de suivi, comme la date de création et le statut initial. Si un utilisateur est connecté, son compte peut être associé à la demande. Si le visiteur n'a pas de compte, les coordonnées du formulaire permettent tout de même de recevoir et traiter le devis.

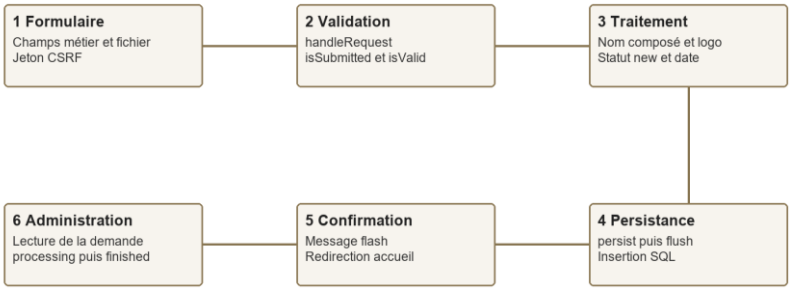


Figure 13 Parcours de traitement d'une demande de devis

Le gestionnaire Doctrine prépare l'enregistrement puis exécute l'écriture en base. La gestion du logo intervient dans ce parcours : le fichier est vérifié, enregistré dans un emplacement privé et sa référence est associée à la demande. Le message de confirmation permet au visiteur de savoir que sa demande a été prise en compte.

J'ai travaillé sur les cas d'erreur autant que sur l'envoi normal. Un formulaire vide, une quantité inférieure à un ou un choix qui ne fait pas partie des valeurs autorisées doivent être refusés. Ces vérifications évitent d'enregistrer des demandes difficiles à exploiter.

Le statut distingue une nouvelle demande, une demande en cours de traitement et une demande terminée. Il aide l'administratrice à suivre son travail. Il ne correspond pas à un paiement ni à une facturation automatique : ces opérations restent hors du périmètre de cette version.

L'extrait ci-dessous montre l'état initial de la demande et l'écriture. persist prépare l'objet pour Doctrine ; flush exécute les opérations nécessaires dans la base. En cas d'échec, le fichier associé est retiré avant de transmettre l'erreur.

Extrait du dépôt • src/Controller/Front/QuoteRequestController.php • lignes 53 à 62

```
53  $quoteRequest->setCreatedAt(new \DateTimeImmutable());
54  $quoteRequest->setStatus(QuoteRequest::STATUS_NEW);
55
56  try {
57      $entityManager->persist($quoteRequest);
58      $entityManager->flush();
59  } catch (\Throwable $exception) {
60      $logos->delete($quoteRequest->getLogo());
61      throw $exception;
62  }
```

Un service dédié au stockage des logos

J'ai regroupé la gestion des fichiers dans QuoteLogoStorage. Le contrôleur coordonne le formulaire et l'enregistrement du devis ; le service s'occupe du fichier. Symfony injecte ce service dans l'action publique et dans le constructeur du contrôleur administratif. Les deux parcours réutilisent ainsi les mêmes règles sans recopier le traitement des uploads.

Le répertoire est défini dans config/services.yaml : private/quote-logos, hors du dossier public. store() vérifie la validité du fichier, sa taille maximale de 15 000 000 octets et la cohérence entre son type MIME et l'image reconnue. Seuls PNG, JPEG et WebP sont admis. Le nom généré avec random_bytes ne reprend pas le nom fourni par le visiteur. Le service demande les permissions 0700 pour le dossier et 0600 pour le fichier.

Du fichier reçu à sa référence en base

Après validation du formulaire, le contrôleur transmet le fichier à store(). La méthode retourne un nom que setLogo() associe au devis. Doctrine conserve cette référence dans la base ; le contenu de l'image reste sur le disque. Le contrôleur définit ensuite la date et le statut avant persist() et flush().

Extraits du dépôt • src/Controller/Front/QuoteRequestController.php • lignes 43 à 45 et 56 à 62

```
if ($logoFile) {
    $quoteRequest->setLogo($logos->store($logoFile));
}
// Après préparation de la date et du statut
try {
    $entityManager->persist($quoteRequest);
    $entityManager->flush();
} catch (\Throwable $exception) {
    $logos->delete($quoteRequest->getLogo());
    throw $exception;
}
```

Si le stockage échoue, le formulaire affiche une erreur et le contrôleur renvoie une réponse 422. Si l'écriture en base échoue après le stockage, il demande la suppression du logo puis transmet l'exception. Cette compensation limite les fichiers sans devis associé ; elle ne constitue pas une transaction commune entre le disque et la base.

Lors d'un remplacement dans l'administration, l'ancien logo est supprimé après la réussite de flush(). En cas d'échec, le nouveau est supprimé et l'ancienne référence rétablie dans l'objet. delete() journalise un avertissement si la suppression échoue. Pour le téléchargement, le contrôleur vérifie ROLE_ADMIN ; le service recherche un fichier autorisé. Cette séparation permet aussi de tester le stockage indépendamment du contrôleur, comme dans QuoteLogoStorageTest.

4.8 Consulter et suivre un devis

L'administration présente les demandes dans une liste. L'administratrice peut ouvrir une demande, lire ses coordonnées et ses caractéristiques puis modifier les informations de suivi prévues par le formulaire administratif.

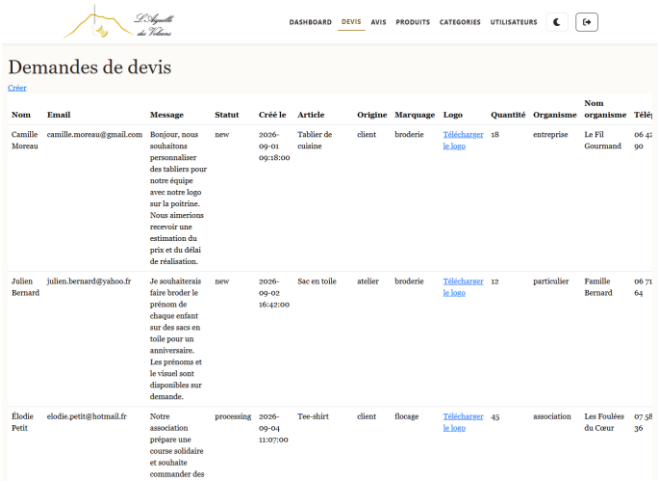


Figure 14 Liste des demandes avec données fictives

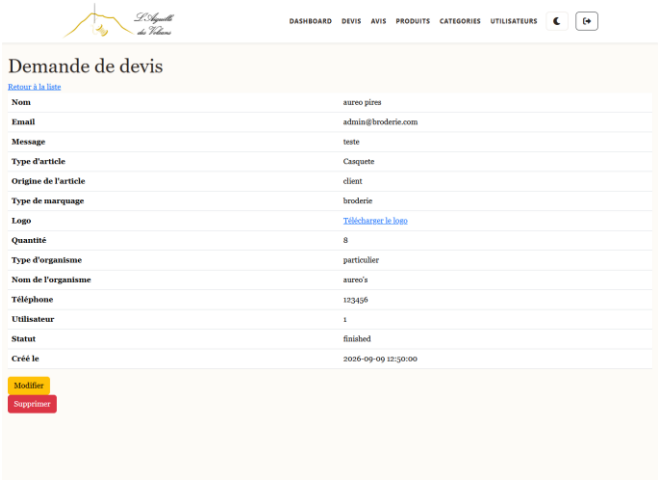


Figure 15 Consultation et traitement d'un devis fictif

La séparation entre le formulaire public et le formulaire administratif est utile. Le visiteur décrit son besoin. L'administratrice traite la demande et met à jour son état. Chaque écran présente les informations adaptées à son utilisateur.

Le logo, lorsqu'il existe, est récupéré depuis une action protégée. Une adresse publique vers le dossier de stockage ne doit pas permettre à un visiteur de consulter les fichiers des autres demandes. Le contrôle du rôle administratif est donc appliqué aussi au téléchargement, pas uniquement au menu de l'administration.

Les captures utilisent les données fictives de la base actuelle. Elles illustrent le fonctionnement du suivi, sans représenter des commandes réellement transmises par des clients.

4.9 Créer lire modifier et supprimer

L'espace d'administration rassemble les fonctions de gestion du site. J'ai développé les opérations souvent appelées CRUD : créer, lire, modifier et supprimer un enregistrement. Dans le catalogue, ces opérations permettent de gérer les réalisations sans modifier directement les fichiers Twig.

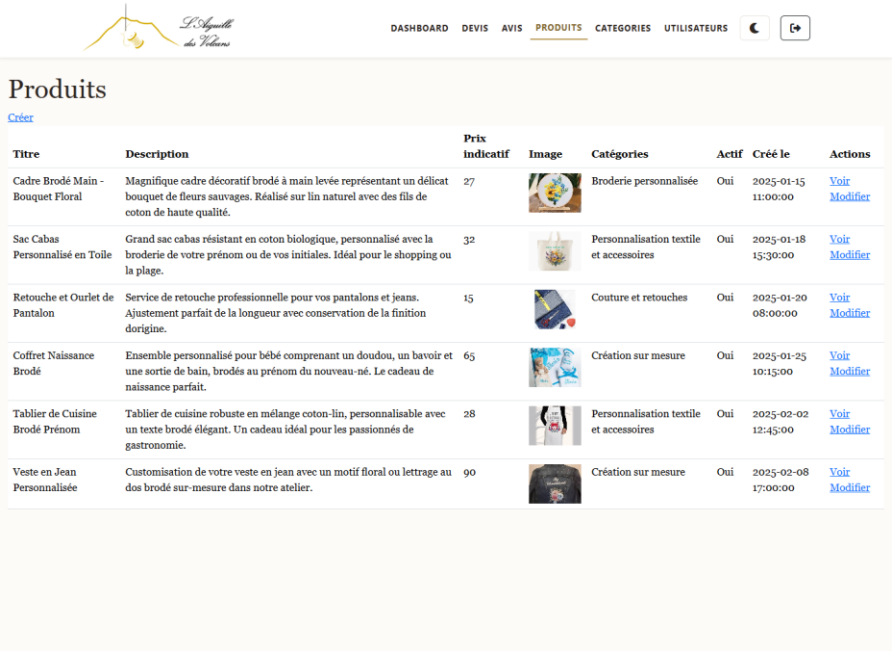


Figure 16 Gestion des produits dans l'administration

La fiche administrative d'un produit contient son titre, sa description, ses catégories, son image, son prix indicatif éventuel et son état d'activation. L'administratrice peut préparer un contenu puis choisir s'il doit être visible. Les catégories servent à regrouper les produits et à faciliter la navigation publique.

Les formulaires de création et de modification transmettent les valeurs à une entité. Le contrôleur vérifie le formulaire puis enregistre les modifications avec Doctrine. Pour une suppression, une action dédiée et un jeton CSRF évitent de traiter une demande qui ne vient pas du formulaire prévu.

J'ai appris à distinguer l'activation d'un produit de sa suppression. Désactiver conserve l'enregistrement mais le retire des listes publiques. Supprimer est une opération différente qui enlève les données concernées et doit tenir compte des relations.

L'administration sert aussi à gérer les avis et les utilisateurs. Ces fonctions partagent une logique commune, mais leurs règles ne sont pas identiques : un avis possède un état de modération, tandis qu'un utilisateur possède des droits d'accès.

4.10 Authentifier les utilisateurs

Le formulaire d'inscription crée un compte avec les informations nécessaires à son utilisation. Le mot de passe saisi est validé puis haché avant l'enregistrement. Le hachage transforme le mot de passe en une valeur de vérification ; le site ne conserve pas le mot de passe lisible.

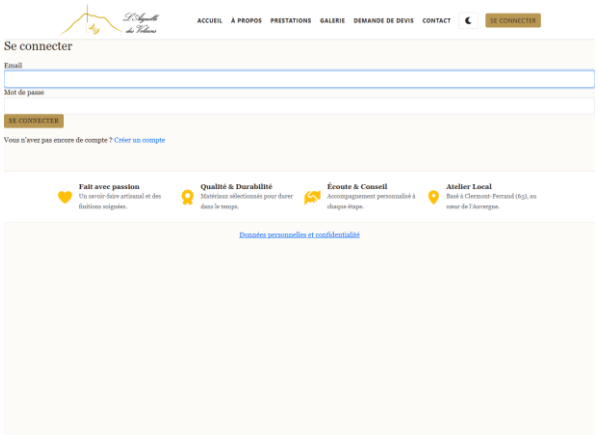


Figure 17 Écran de connexion

Symfony utilise le compte correspondant à l'adresse de courriel pour vérifier la connexion. Une fois connecté, l'utilisateur est reconnu dans la session. Le rôle ROLE_USER correspond aux usages d'un compte ordinaire. ROLE_ADMIN permet l'accès à l'administration.

Les droits sont vérifiés sur le serveur. Masquer un bouton dans Twig ne suffirait pas à protéger une action : une personne pourrait essayer d'ouvrir directement son adresse. La configuration de sécurité et les contrôles des actions empêchent cet accès sans le rôle requis.

La connexion utilise une protection CSRF et une limitation des tentatives configurée à cinq tentatives par minute. Pour les nouveaux mots de passe, le projet impose une longueur minimale de douze caractères. Ces mesures apportent des protections adaptées à un site qui contient des coordonnées et un espace de gestion.

Le projet comprend un mécanisme préparé pour les courriels de vérification. La messagerie est actuellement désactivée tant que la boîte et le transport ne sont pas configurés. Je distingue donc la connexion fonctionnelle des notifications qui seront activées après cette configuration.

L'adresse du compte est unique. Le formulaire et les règles de l'entité permettent de contrôler la création avant d'enregistrer les données. Pour le hachage, le traitement utilise le service prévu par Symfony.

Extrait du dépôt • src/Controller/Front/RegistrationController.php • lignes 48 à 51

```
48  $user->setPassword($userPasswordHasher->hashPassword($user, $plainPassword));
49
50  $entityManager->persist($user);
51  $entityManager->flush();
```

4.11 Modération des avis

Les avis apportent des retours sur les prestations et les réalisations. Pour déposer un avis, l'utilisateur doit être connecté. Le traitement associe le témoignage à son compte, enregistre une note et un contenu puis place l'avis dans un état d'attente.

Le formulaire contrôle notamment la note, qui doit rester entre un et cinq. La date et le statut de création sont préparés par l'application. Un visiteur ne doit pas pouvoir choisir lui-même que son avis est déjà approuvé.

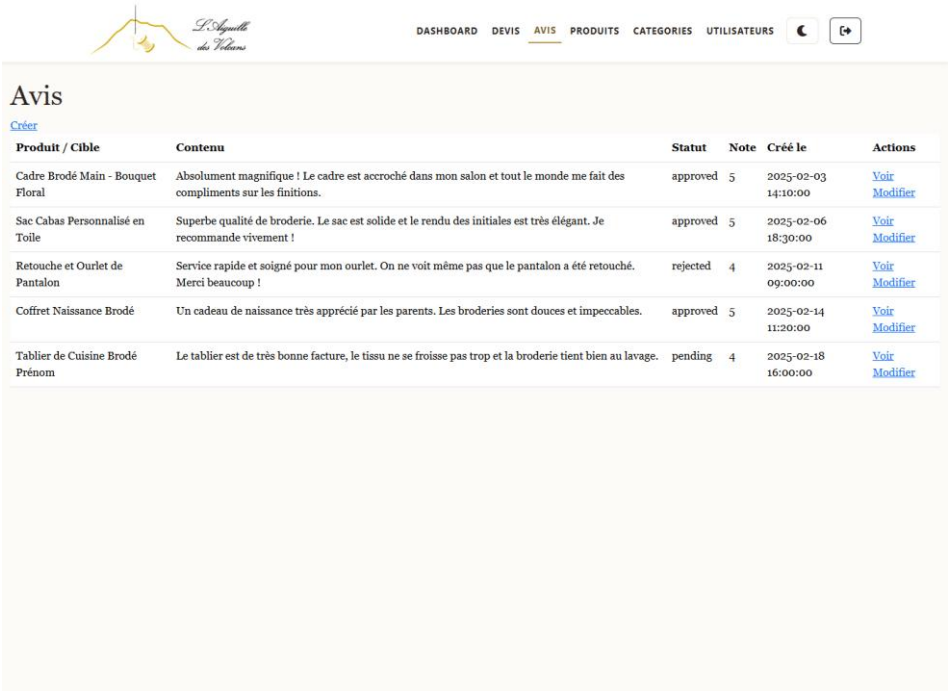


Figure 18 Modération des avis avec données fictives

L'administratrice consulte les avis reçus et peut les approuver ou les refuser. Le site public ne présente que les avis approuvés. Cette règle permet de vérifier le contenu avant de le rendre visible à tous.

L'affichage public utilise le prénom de l'auteur pour limiter les informations personnelles publiées. L'adresse électronique du compte n'est pas une information à montrer à côté d'un témoignage.

Cette fonctionnalité m'a permis de travailler plusieurs notions ensemble : la relation entre un avis et un utilisateur, la validation des données, la création d'un état initial et la sélection des éléments autorisés à apparaître. Elle illustre qu'un enregistrement en base ne signifie pas automatiquement une publication immédiate.

5 Développement du front-end

5.1 Construire les pages avec Twig

Twig permet d'utiliser un modèle commun pour les pages. Le fichier de base rassemble notamment la structure HTML, les ressources, la navigation et le pied de page. Chaque page apporte ensuite le contenu propre à son objectif.

Cette organisation évite de recopier le menu sur toutes les pages. Une modification de la navigation peut être faite à un seul endroit. Les templates peuvent utiliser des conditions, par exemple pour afficher un accès réservé aux personnes connectées, et des boucles pour parcourir les produits transmis par le contrôleur.

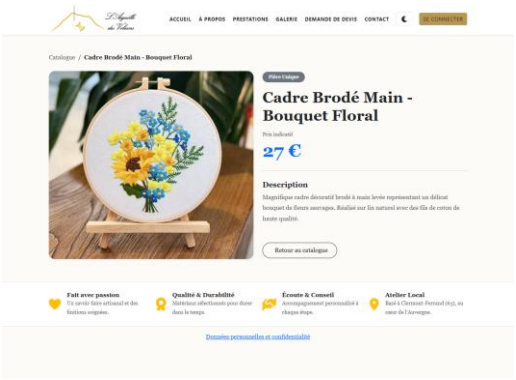


Figure 19 Fiche d'une réalisation et contenu issu de la base

Une fiche produit affiche les données d'un enregistrement : titre, image, description et autres informations disponibles. Le template ne choisit pas seul les produits à lire dans MariaDB. Il reçoit le résultat du traitement côté serveur et définit la manière de le présenter.

Twig échappe les valeurs affichées par défaut. Un texte provenant d'un formulaire est donc traité comme du texte dans les usages ordinaires, au lieu d'être directement exécuté comme une instruction HTML. Je veille à conserver ce fonctionnement pour les contenus saisis.

Pour apprendre à corriger les pages, j'ai suivi les données d'une étape à l'autre : nom de la variable dans le contrôleur, valeur envoyée au rendu puis variable utilisée dans Twig. Cette méthode m'a aidé à comprendre la communication entre back-end et front-end.

Dans cet extrait, la boucle crée un lien pour chaque catégorie reçue. path construit l'adresse de la route à partir de son nom et du slug. Le template peut ainsi présenter des catégories qui évoluent dans la base.

Extrait du dépôt • templates/front/product/index.html.twig • lignes 16 à 21

```
16  {% for category in categories %}
17      <a href="{{ path('app_category_show', {slug: category.slug}) }}"
18         class="btn btn-outline-dark btn-sm rounded-pill px-3 py-2">
19          {{ category.name }}
20      </a>
21  {% endfor %}
```

5.2 Présenter les réalisations

La galerie présente les réalisations sous forme de cartes. Chaque carte regroupe une image et les informations nécessaires pour identifier un produit. Le visiteur peut ouvrir une fiche pour obtenir plus de détails ou consulter une catégorie pour réduire la liste.

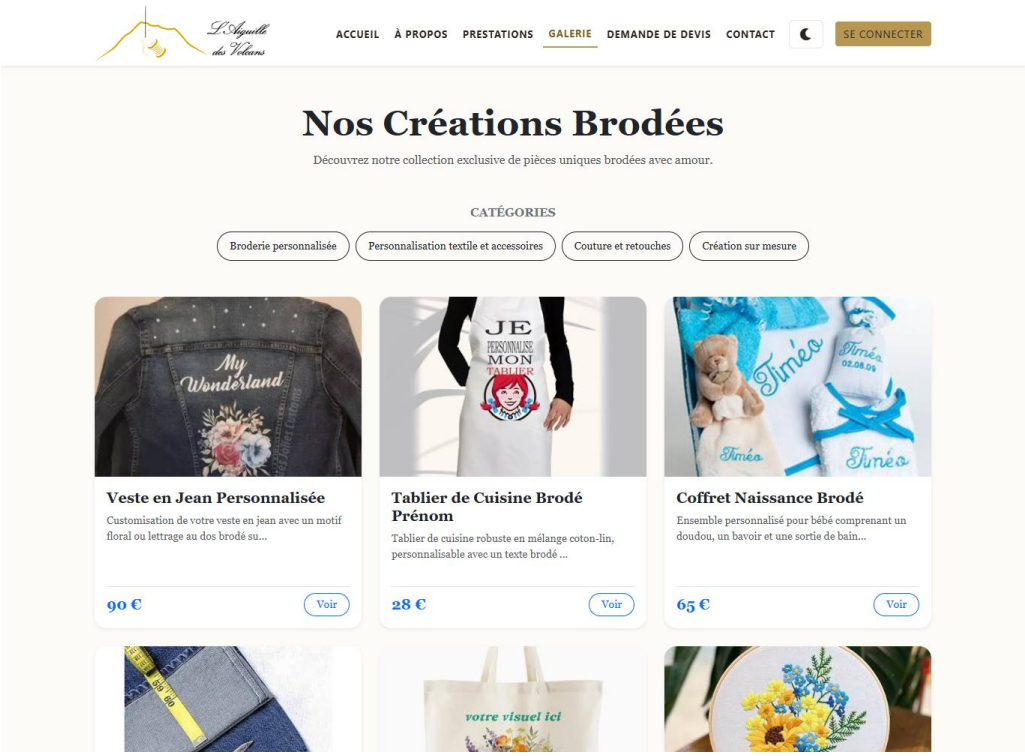


Figure 20 Galerie des produits actifs

Bootstrap apporte une grille qui adapte le nombre de colonnes à la largeur. Le CSS personnalisé complète cette base pour les couleurs, les espacements, les boutons et la présentation des cartes. Le résultat garde une cohérence avec les pages de services et les formulaires.

Les éléments réutilisables permettent de maintenir une présentation commune. Lorsque plusieurs cartes sont générées dans une boucle Twig, leur contenu varie mais leur structure reste la même. Cela rend les pages plus simples à entretenir et évite des différences de présentation entre produits.

La règle de visibilité est appliquée avant le rendu. Une réalisation inactive ne doit pas apparaître dans les listes publiques. L'administration peut continuer à la montrer pour permettre sa modification. La même donnée a donc une utilisation différente selon l'écran et les droits de la personne.

Les images sont importantes pour ce catalogue. Leur disposition doit permettre de comprendre le travail présenté sans masquer les boutons ni rendre le texte difficile à lire sur un écran étroit.

5.3 Rendre les interfaces utilisables

J'ai contrôlé les pages à une largeur d'ordinateur et à une largeur de téléphone. La grille passe d'une présentation en colonnes à une lecture verticale. Le menu et les formulaires s'adaptent à l'espace disponible. La vérification concerne aussi les messages d'erreur, car ils doivent rester lisibles au moment où l'utilisateur en a besoin.



Figure 21 Accueil à une largeur de téléphone de 390 pixels

Pour l'accessibilité, je m'appuie sur des principes simples : une langue française déclarée, des titres structurés, des libellés pour les champs et des boutons identifiables. Le projet comporte un lien pour aller directement au contenu et des noms accessibles pour les commandes du menu. Les images informatives doivent être accompagnées d'un texte alternatif adapté.

L'utilisation au clavier fait également partie des points à regarder. Le focus doit rester visible. Lorsqu'une image est agrandie, la fermeture de cette vue et le retour du focus sont pris en compte. Ces détails aident les personnes qui ne naviguent pas avec une souris.

Ces mesures s'inscrivent dans les principes du RGAA. Je présente les dispositions réalisées et les vérifications du projet ; je ne revendique pas un audit complet de conformité. La poursuite des contrôles sur plusieurs appareils reste une habitude de maintenance du site.

5.4 Ajouter des comportements dans le navigateur

J'utilise JavaScript pour répondre aux actions du visiteur dans le navigateur : changement de thème, agrandissement des images et aperçu d'un fichier. Pour détailler cette démarche, je prends le thème clair et sombre. Son code se trouve dans assets/scripts/theme.js, importé par assets/scripts/app.js.

Repérer la commande et retrouver le choix mémorisé

Dans le template du menu, le bouton porte l'attribut data-theme-toggle. Ce repère permet de sélectionner les commandes sans dépendre de leur apparence. À l'événement DOMContentLoaded, le script lit la clé site-theme dans localStorage. Si sa valeur est dark, setTheme() active le thème sombre ; sinon, le thème clair est utilisé. La préférence est conservée dans ce navigateur pour la même origine, sans être enregistrée dans le compte utilisateur.

Extrait du dépôt • assets/scripts/theme.js • lignes 31 à 43

```
document.addEventListener('DOMContentLoaded', () => {
    const savedTheme = localStorage.getItem(themeStorageKey);
    setTheme(savedTheme === darkTheme ? darkTheme : lightTheme);

    document.querySelectorAll('[data-theme-toggle]').forEach((button) => {
        button.addEventListener('click', () => {
            const isDark = document.body.classList.toggle('dark-mode');
            const theme = isDark ? darkTheme : lightTheme;
            localStorage.setItem(themeStorageKey, theme);
            updateThemeButton(isDark);
        });
    });
});
```

Capturer le clic et changer l'état

querySelectorAll() récupère tous les boutons concernés. forEach() attache un écouteur click à chacun. Au clic, classList.toggle() ajoute ou retire dark-mode sur body et retourne un booléen. Je réutilise ce résultat pour choisir la valeur dark ou light, la mémoriser avec localStorage.setItem(), puis mettre à jour les commandes. Aucune requête au serveur n'est nécessaire pour ce changement.

Le script modifie donc un état simple : la présence de la classe dark-mode. Le CSS utilise body.dark-mode pour redéfinir les variables de couleur, notamment celles du fond et du texte. Les composants qui utilisent ces variables changent d'apparence tout en conservant leur contenu et leur structure.

Synchroniser les commandes et les images du thème

updateThemeButton() actualise tous les boutons : aria-label décrit l'action suivante, title indique le mode proposé et les classes fa-moon et fa-sun changent l'icône. Lorsque le mode sombre est actif, le libellé devient « Activer le mode clair ». Le bouton reste ainsi compréhensible au-delà de son seul pictogramme.

Extrait du dépôt • assets/scripts/theme.js • lignes 11 à 23

```
const updateThemeButton = (isDark) => {
    document.querySelectorAll('[data-theme-toggle]').forEach((button) => {
        const icon = button.querySelector('i');
        button.setAttribute('aria-label', isDark ? 'Activer le mode clair' : 'Activer le mode
sombre');
        button.setAttribute('title', isDark ? 'Mode clair' : 'Mode sombre');

        if (icon) {
            icon.classList.toggle('fa-moon', !isDark);
            icon.classList.toggle('fa-sun', isDark);
        }
    });
    updateThemeLogos(isDark);
};
```

La fonction appelle ensuite updateThemeLogos(). Les images repérées par data-theme-logo reçoivent la variante claire ou sombre de leur source. Lors du chargement d'une autre page, setTheme() applique le choix mémorisé puis appelle la même fonction de mise à jour. Le clic et l'initialisation réutilisent ainsi les mêmes règles pour les boutons et les logos.

Vérifier le comportement du script

Une vérification complémentaire du fichier JavaScript a été exécutée le 17 octobre 2026 avec un DOM simulé. Elle couvre l'absence de préférence, le passage au sombre, la restauration du choix lors d'un nouveau chargement, le retour au clair et une valeur mémorisée inconnue. Les assertions ont confirmé l'état de la classe, la valeur stockée, les libellés des boutons et les sources des logos attendus.

Cette vérification contrôle la logique du script ; elle ne valide pas le rendu des couleurs, le chargement réel des images ou la navigation au clavier. Dans le navigateur, ces points se contrôlent sur plusieurs pages, après rechargement et aux largeurs ordinateur et mobile. Ce contrôle complémentaire est distinct de la suite PHP présentée dans la section 7.1.

Le choix technique reste adapté à cette interaction : JavaScript gère l'événement et l'état, le CSS définit l'apparence et localStorage mémorise la préférence. Cette mémorisation est locale au navigateur et ne synchronise pas le thème entre plusieurs appareils.

6 Sécurité et confidentialité

6.1 Les protections appliquées

Le site contient des comptes et des coordonnées de demandeurs. J'ai appliqué des protections aux points où les données entrent, sont conservées et peuvent être consultées. Les outils Symfony et Doctrine apportent une base, mais il faut les utiliser correctement dans les formulaires et les actions.

Point contrôlé	Mesure dans le projet	Utilité
Mot de passe	Hachage et longueur minimale	Ne pas conserver le mot de passe lisible
Connexion	CSRF et limitation des tentatives	Limiter les demandes de connexion abusives
Administration	Rôle ROLE_ADMIN vérifié sur le serveur	Réserver les modifications aux personnes autorisées
Formulaire	Contraintes Symfony	Refuser les valeurs incohérentes ou manquantes
Requête de données	Paramètres Doctrine	Séparer les valeurs saisies de l'expression de recherche
Affichage de texte	Échappement Twig	Éviter l'interprétation directe des textes saisis comme du HTML
Suppression et modification	Jetons CSRF selon les formulaires	Contrôler l'origine de l'action reçue

Le contrôle côté serveur est essentiel. Les restrictions du navigateur peuvent aider l'utilisateur, mais une demande peut être envoyée sans passer par l'interface habituelle. Le contrôleur doit donc appliquer les mêmes règles avant de modifier la base.

J'ai aussi séparé la partie publique de l'administration. Le visiteur peut découvrir des réalisations sans accéder aux coordonnées des devis. Les tests vérifient les refus d'accès, les saisies invalides et les actions qui nécessitent une connexion.

Ces protections répondent aux besoins de la première version. Leur vérification fait partie du développement : je ne me contente pas de voir qu'un envoi correct fonctionne, je contrôle également ce que fait le site quand une demande est incorrecte ou non autorisée.

6.2 Protéger les fichiers de devis

Un logo transmis avec un devis peut contenir un élément destiné à un projet précis. Il n'a pas vocation à être disponible dans la galerie publique. J'ai donc séparé son stockage de celui des images de présentation du site.

Le formulaire accepte des images PNG, JPEG et WebP avec une limite de taille de quinze mégaoctets. Le serveur vérifie le fichier reçu, notamment son type et sa capacité à être reconnu comme une image. Le nom de stockage est généré par l'application plutôt que repris directement du nom donné par le visiteur.

Le fichier est placé dans un dossier privé. La base conserve sa référence afin de le retrouver depuis la demande. Pour le télécharger, l'administratrice utilise une route protégée qui vérifie ses droits avant de renvoyer le fichier. Les réponses de téléchargement comportent aussi des indications pour limiter la mise en cache et empêcher une interprétation incorrecte du type de contenu.

Étape	Contrôle ou comportement
Sélection du fichier	Aperçu éventuel dans le navigateur
Réception du formulaire	Vérification de la taille et des formats autorisés
Enregistrement	Nom généré et stockage hors de l'accès public
Consultation	Téléchargement réservé à l'administration
Remplacement ou suppression	Retrait de l'ancien fichier après l'opération réussie

Cette fonctionnalité m'a appris qu'un upload ne consiste pas seulement à déplacer un fichier. Il faut décider où il sera conservé, qui pourra le consulter et ce qu'il deviendra lorsque la demande est modifiée ou supprimée.

Les vérifications automatiques couvrent les fichiers acceptés, les fichiers refusés et les droits de téléchargement. Elles permettent de retrouver rapidement une erreur si le formulaire ou le stockage change dans une prochaine version.

6.3 Informer les utilisateurs

J'ai ajouté une page de confidentialité accessible depuis le site. Elle décrit les informations utilisées pour les comptes, les demandes de devis, les contacts et les avis. La responsable indiquée est Oceane Roux, pour L'Aiguille des Volcans, à 4 Route de Clermont, 63530 Volvic.

Pour une demande de devis, les coordonnées et les informations du projet servent à préparer une réponse et à suivre les échanges. Les données de contact et de devis peuvent être conservées jusqu'à trois ans après le dernier contact. Le logo est retiré lorsqu'il n'est plus nécessaire. Cette gestion est actuellement une procédure à appliquer dans l'administration ; elle n'est pas présentée comme une purge automatique.

Le visiteur peut demander l'accès, la rectification ou la suppression de ses données selon sa situation. La page indique contact@laiguilledesvolcans.fr comme adresse prévue pour ces demandes. Tant que la boîte est en préparation, les coordonnées postales et téléphoniques de l'atelier offrent aussi un moyen de contact.



Figure 22 Page de confidentialité de la version locale

Pour guider les choix, j'ai consulté les principes présentés par la CNIL et la documentation technique. Ma veille porte notamment sur la validation des fichiers, les protections des formulaires et les mises à jour des dépendances. La documentation Symfony explique l'utilisation du CSRF ; les recommandations OWASP sur les uploads m'aident à vérifier les formats, les noms et l'emplacement des fichiers.

Je garde les liens utiles en annexe. Cette démarche me permet de revenir à une source de référence lorsqu'une règle ou un outil évolue, plutôt que de conserver une solution que je ne comprends pas.

7 Vérification du site

7.1 Les tests et leurs résultats

Pendant le développement, j'ai vérifié manuellement les principaux parcours du site, notamment la navigation, les formulaires et l'affichage sur différentes tailles d'écran. En complément, une suite de tests automatisés a été exécutée sur la version locale. Ses résultats ne constituent pas un décompte de mes essais manuels.

Le 18 octobre 2026 à 14 h 36, heure de Paris, trois contrôles complémentaires ont été exécutés dans l'environnement local. PHPUnit : 63 tests et 229 assertions réussis. Doctrine : mappage des entités valide et schéma de la base synchronisé. Twig : syntaxe valide pour les 54 templates contrôlés. Les rapports complets sont conservés dans output/tests, sous les noms phpunit-2026-10-18_14-36-39.txt, doctrine-2026-10-18_14-36-39.txt et twig-2026-10-18_14-36-39.txt. Ces résultats portent sur les scénarios automatisés, la structure des données et la syntaxe ; ils ne remplacent pas les vérifications visuelles du site.

Jeu d'essai ou situation	Résultat attendu	Résultat constaté
Devis avec champs requis absents	Formulaire invalide et aucun enregistrement	Test réussi
Quantité égale à zéro	Refus de la quantité	Test réussi
Choix transmis hors des valeurs proposées	Refus de la valeur	Test réussi
Fichier image autorisé et valide	Acceptation par le contrôle de fichier	Test réussi
Fichier interdit ou trop volumineux	Refus du fichier	Test réussi
Téléchargement de logo sans rôle administratif	Accès refusé	Test réussi
Produit inactif dans une catégorie	Absence des résultats publics	Test réussi
Contact valide avec messagerie désactivée	Message d'indisponibilité et saisie conservée	Test réussi

Une assertion vérifie un résultat précis, par exemple qu'une réponse est refusée ou qu'une valeur n'est pas présente. Le nombre de tests ne remplace pas le contrôle visuel : une page peut être techniquement valide mais difficile à lire sur téléphone.

Les deux approches se complètent : mes essais manuels portent sur les écrans et leur utilisation, tandis que les tests automatisés vérifient les règles et les scénarios définis dans le code. Leur réussite ne garantit pas l'absence de toute anomalie et ne remplace pas une vérification visuelle.

Exécution documentée et exemple de validation

La commande ci-dessous exécute la suite PHPUnit existante dans le service PHP de Docker. Les contrôles Doctrine et Twig ont été lancés séparément avec doctrine:schema:validate et lint:twig templates. Chaque rapport texte conserve la sortie complète du contrôle correspondant.

```
docker compose exec -T php php bin/phpunit --testdox --colors=never
  Contrôles locaux — 18 octobre 2026, 14 h 36 (Paris)
  PHPUnit : php bin/phpunit --testdox --colors=never
  OK (63 tests, 229 assertions)

  Doctrine : php bin/console doctrine:schema:validate --no-ansi
  [OK] The mapping files are correct.
  [OK] The database schema is in sync with the mapping files.

  Twig : php bin/console lint:twig templates --no-ansi
  [OK] All 54 Twig files contain valid syntax.
```

Extraits des sorties réelles des trois contrôles du 18 octobre 2026 à 14 h 36, reproduits depuis les rapports texte. Les lignes intermédiaires sont omises.

Règle vérifiée et préparation

Une demande de devis doit porter sur au moins un article. QuoteRequestTypeTest utilise des données fictives valides, dont une quantité de douze polos. Le fournisseur invalidData() transmet ensuite le cas « quantité zéro » : field vaut quantity et value vaut 0. Le test remplace uniquement cette valeur avant de soumettre le formulaire Symfony.

Extraits de tests/Form/QuoteRequestTypeTest.php • cas de données et assertions

```
yield 'quantité zéro' => ['quantity', 0];
```

```
$form->submit(array_replace(self::VALID_DATA, [$field => $value]));
self::assertFalse($form->isValid());
self::assertGreaterThan(0, count($form->get($field)->getErrors(true)));
```

Action et résultats attendus

submit() déclenche le traitement et la validation du formulaire. La première assertion exige que isValid() soit faux. La seconde exige au moins une erreur sur le champ quantity. Le scénario « quantité zéro » a réussi : ces deux conditions sont vérifiées. Le même test est réutilisé pour d’autres données invalides, ce qui explique plusieurs scénarios à partir d’une seule méthode.

Portée du test

Ce test vérifie la validation côté serveur, sans navigateur visuel. Le CSRF est désactivé uniquement dans ce test isolé pour cibler les contraintes des champs. Il ne vérifie pas une écriture en base : le scénario fonctionnel distinct testInvalidQuoteDoesNotPersist() contrôle qu’une demande avec un e-mail invalide et un prénom vide n’est pas enregistrée.

7.2 Difficultés rencontrées et solutions apportées

La première difficulté a été de répondre aux besoins d'une autre personne. Une fonctionnalité ne doit pas seulement être intéressante à programmer ; elle doit être utile à l'activité. Le formulaire de devis m'a aidé à relier les champs aux informations nécessaires pour préparer une réponse commerciale.

J'ai également dû comprendre l'environnement Symfony avec Docker. Au début, plusieurs outils intervenaient dans une même page. J'ai progressivement identifié leur rôle : le serveur web reçoit la demande, PHP exécute Symfony et MariaDB fournit les données. Cette distinction m'a permis de mieux situer un problème avant de modifier le code.

Les relations Doctrine ont demandé un travail particulier. La relation entre produits et catégories m'a fait comprendre pourquoi une table de liaison est nécessaire. Le lien facultatif entre un devis et un utilisateur m'a montré qu'un visiteur peut transmettre une demande sans posséder de compte.

La communication entre contrôleur, repository et Twig a été une autre étape importante. Pour vérifier un affichage, je suis les données depuis la recherche jusqu'au template. Cette méthode est plus efficace que de modifier plusieurs fichiers sans savoir à quel endroit se situe le problème.

Enfin, l'intégration responsive m'a demandé de comparer la maquette à des largeurs réelles. Une disposition satisfaisante sur ordinateur peut produire un formulaire trop serré sur téléphone. Le regroupement vertical des champs et la grille Bootstrap m'ont aidé à adapter les écrans.

Pour résoudre ces difficultés, j'ai consulté la documentation et demandé des explications aux enseignants et aux collègues du cours. J'ai ensuite vérifié les changements dans l'application. Le résultat est un projet que je peux présenter à partir de parcours concrets : consulter une réalisation, envoyer un devis et le retrouver dans l'administration.

Un exemple concret de correction concerne les catégories : toutes les entrées publiques doivent exclure les produits inactifs. J'ai regroupé la logique de recherche dans le repository et vérifié les routes concernées. Pour le devis, les contrôles de saisie ont été renforcés afin que la validation ne dépende pas seulement des attributs HTML. Ces corrections sont illustrées par les extraits et les tests présentés plus haut.

8 Publication du site

8.1 La première mise en ligne sur OVH

Une première version est déjà accessible à l'adresse www.laiguilledesvolcans.fr sur un hébergement OVH. J'ai transféré les fichiers avec FileZilla. Pour la base, j'ai exporté la base locale puis importé le fichier SQL avec phpMyAdmin sur l'hébergement.

Le dossier public constitue l'entrée web de Symfony. La publication doit permettre au serveur de recevoir les demandes par cette entrée, tout en conservant les fichiers internes de l'application hors de l'accès direct du navigateur. Les paramètres de l'hébergement, notamment la connexion à la base, doivent correspondre à l'environnement distant.

Le site public est actuellement en mode maintenance. La page de connexion reste accessible et un compte administratif permet de consulter l'application. Cela permet de préparer l'ouverture du site et de contrôler les fonctions avant de donner accès à l'ensemble des pages aux visiteurs.

Étape de publication	Vérification associée
Préparer les fichiers et les ressources compilées	Présence des dépendances et de public/build
Sauvegarder la version distante et sa base	Possibilité de revenir à la version précédente
Transférer les fichiers avec FileZilla	Fichiers arrivés dans les dossiers prévus
Configurer l'environnement distant	Accès à MariaDB et paramètres de production
Importer ou faire évoluer la base avec phpMyAdmin	Structure et enregistrements disponibles
Contrôler le site en maintenance	Pages, connexion, catalogue et administration
Préparer l'ouverture publique	Dernières vérifications des parcours et de la messagerie

Les dernières corrections présentées dans ce dossier sont validées localement. Leur transfert sur l'hébergement est prévu avant la soutenance. Pour cette mise à jour, je conserve une sauvegarde et je prends en compte les données déjà présentes au lieu de remplacer la base sans contrôle.

La messagerie sera activée après création de la boîte et configuration du transport. Le webmail, tel que Roundcube, sert à consulter les messages ; le site a aussi besoin de paramètres d'envoi pour transmettre ses notifications.

9 Bilan du projet

9.1 Le résultat et les prochaines étapes

L'Aiguille des Volcans m'a permis de construire une application qui relie la conception, le front-end, le back-end et une base relationnelle. Le visiteur peut découvrir l'activité, parcourir des réalisations et transmettre une demande de devis. L'administratrice peut gérer les contenus, modérer les avis et suivre les demandes dans un espace réservé.

Le résultat principal pour moi est de comprendre le trajet des données. Une information saisie dans un formulaire est validée, enregistrée puis retrouvée dans un autre écran. Les règles de visibilité et les droits d'accès permettent de présenter ces informations à la bonne personne.

J'ai aussi appris à utiliser les erreurs comme des points de vérification. Un fichier refusé, une quantité incorrecte ou un accès interdit font partie du comportement attendu du site. Les tests et les contrôles dans le navigateur m'aident à vérifier ces situations après une modification.

Les prochaines étapes sont proportionnées aux besoins de l'activité : transférer la dernière version, activer la messagerie et effectuer les derniers essais sur l'hébergement. Ensuite, la pagination des listes et l'amélioration de la recherche pourront devenir utiles si le catalogue et les demandes augmentent. Une traduction pourra être envisagée selon le public visé.

Sur l'organisation, je souhaite reprendre les issues et milestones pour mieux suivre les tâches à venir, tout en conservant l'habitude des commits et push réguliers. Cette amélioration concerne ma méthode de travail et m'aidera à mieux préparer les prochaines versions.

Ce projet constitue une première expérience complète de développement pour une activité réelle. Je peux expliquer mes choix avec les fonctionnalités réalisées, les schémas, les extraits de code et les résultats des vérifications. Il représente une base que je pourrai faire évoluer en continuant à apprendre.

Présentation courte en anglais

L'Aiguille des Volcans is a website for an embroidery and textile customisation business. Visitors can discover the services, browse products and request a quote. The administrator can manage the catalogue, review customer feedback and follow quote requests. I developed the application with Symfony, Twig, Doctrine and MariaDB. This project helped me understand how a form, a database and a web page work together.

Annexe A Maquette du devis sur ordinateur

Maquette issue des exports graphiques du projet Figma. Elle présente l'intention de conception du formulaire avant les ajustements de l'intégration. Le fonctionnement actuel est présenté dans les captures des chapitres de développement.

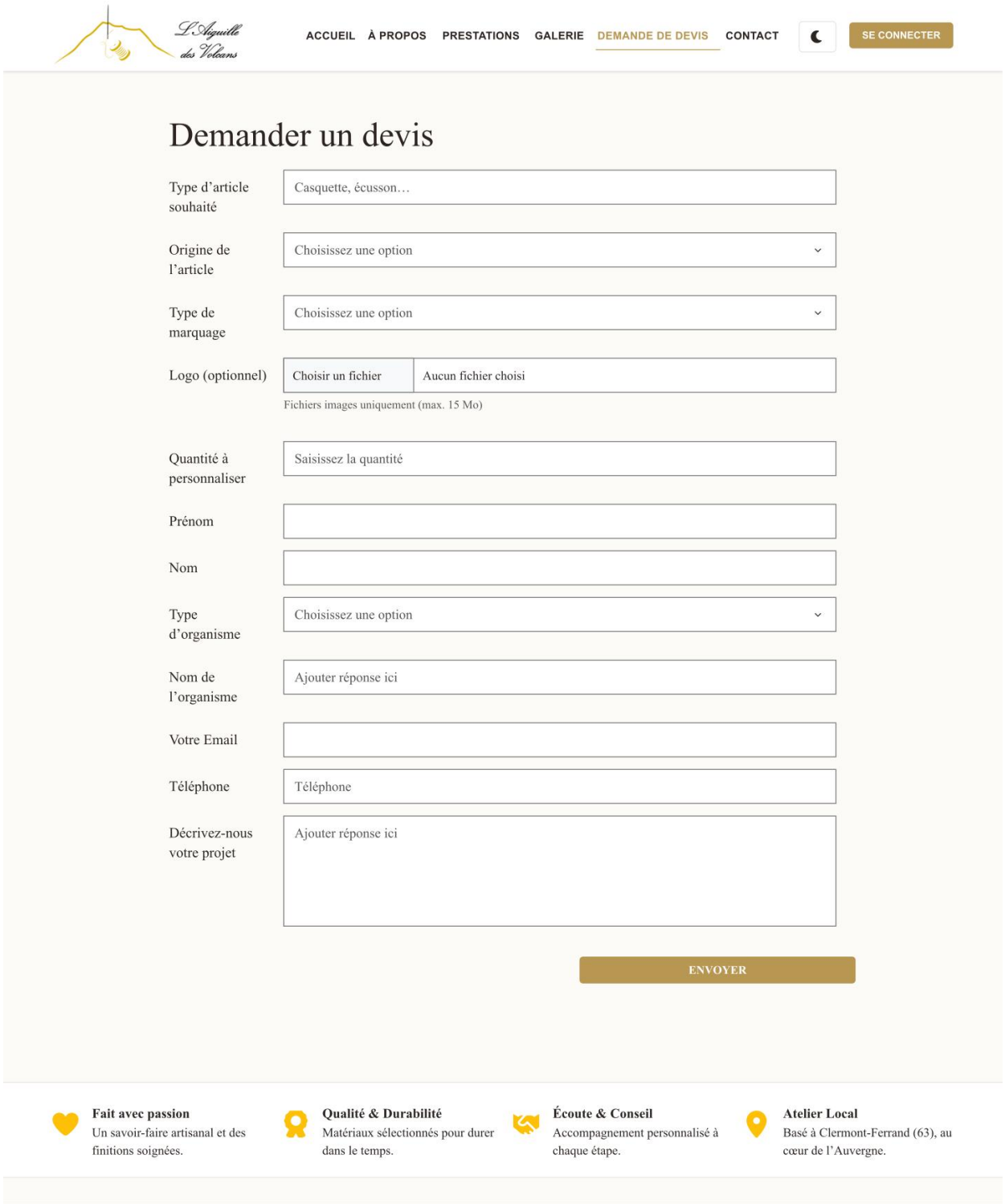


Figure A1 Maquette complète du devis sur ordinateur

Annexe B Maquette du devis sur mobile

Les deux parties se lisent de gauche à droite et correspondent à la progression verticale du formulaire. Les champs et les choix ont ensuite été reliés au formulaire Symfony.

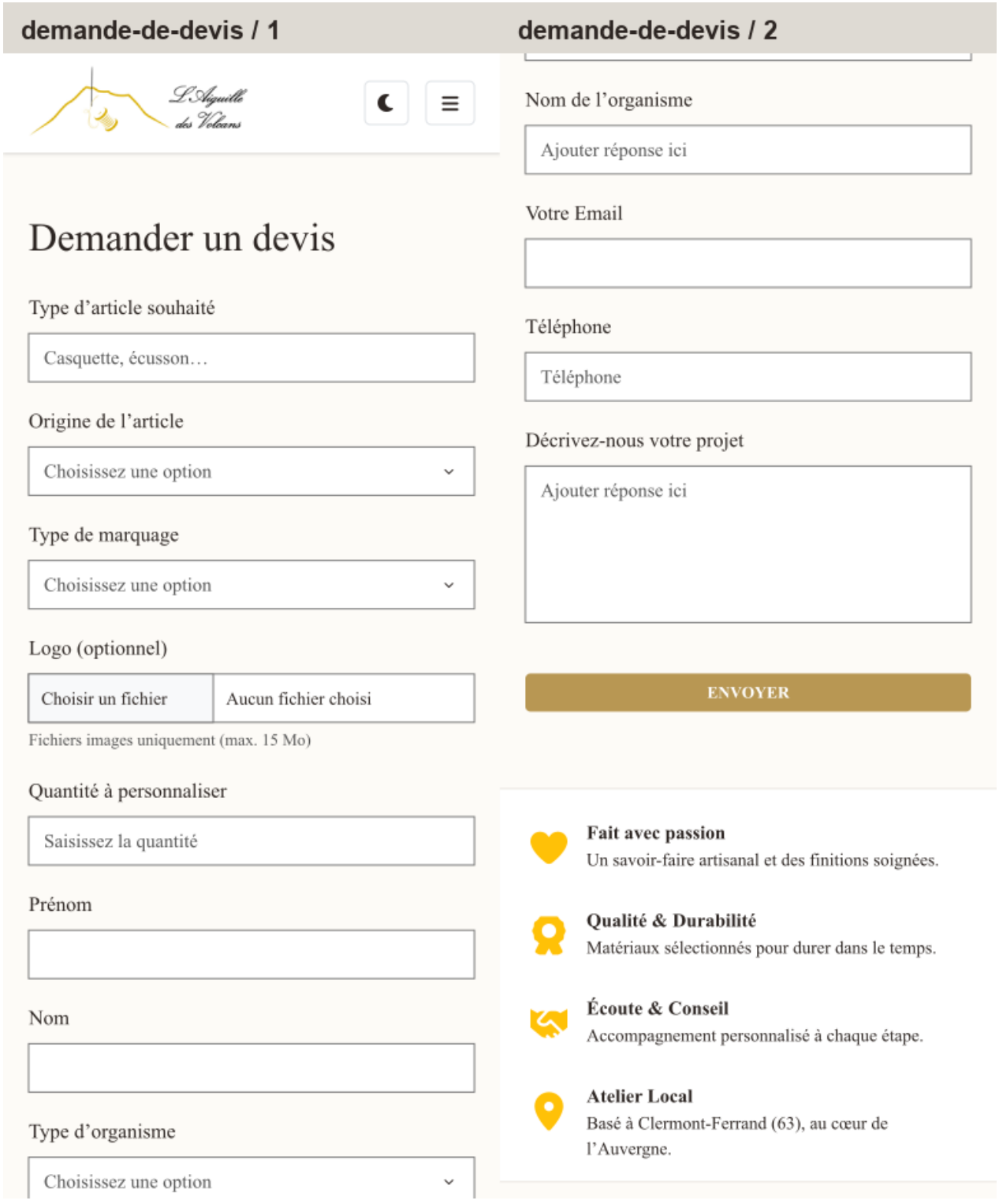


Figure B1 Maquette mobile du devis en deux parties

Annexe C Sources et repères techniques

Les références ci-dessous servent à comprendre le cadre du titre et les outils utilisés. Le dossier d'exemple a été utilisé comme repère de structure, avec un contenu entièrement adapté à L'Aiguille des Volcans.

Référentiel DWWM 2023 TP 01280 millésime 04 et référentiel d'évaluation fournis dans les supports de formation.

Fiche RNCP 37674 France compétences
https://www.francecompetences.fr/recherche/rncp/37674/

Documentation officielle Symfony
https://symfony.com/doc/7.4/forms.html

https://symfony.com/doc/7.4/security.html

https://symfony.com/doc/7.4/security/csrf.html

Documentation Doctrine ORM
https://www.doctrine-project.org/projects/doctrine-orm/en/3.6/index.html

CNIL Principes de protection des données
https://www.cnil.fr/fr/les-six-grands-principes-du-rgpd

RGAA — Référentiel général d'amélioration de l'accessibilité (DINUM)
https://accessibilite.numerique.gouv.fr/

OWASP File Upload Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Dépôt du projet
https://github.com/aureopires/projet_broderie

Maquettes Figma
https://www.figma.com/design/oWgNwwsssc6BPxEEJrHwZl/L-Aiguille-des-Volcans

Repères de vocabulaire

Route : adresse associée à une action. Repository : classe qui rassemble les recherches de données. ORM : outil qui relie des objets aux tables. Migration : évolution enregistrée de la structure SQL. CSRF : protection des actions contre une demande forgée depuis un autre site. Responsive : adaptation de la disposition à la largeur de l'écran. Slug : identifiant lisible utilisé dans une adresse.

Annexe D Exemple SQL et jeu d’essai

La création de Category est conservée dans la migration Version20260729074248. Voici sa commande SQL, remise en lignes pour la lecture. Elle illustre une clé primaire et des champs facultatifs. Les autres migrations complètent la structure et les relations.

```
CREATE TABLE category (
    id INT AUTO_INCREMENT NOT NULL,
    name VARCHAR(255) NOT NULL,
    image VARCHAR(255) DEFAULT NULL,
    slug VARCHAR(255) NOT NULL,
    description LONGTEXT DEFAULT NULL,
    PRIMARY KEY (id)
) DEFAULT CHARACTER SET utf8mb4;
```

La table product_category relie un produit à plusieurs catégories. Sa clé primaire composée évite de répéter deux fois la même association. Les clés étrangères définies dans la migration relient ses identifiants aux tables Product et Category.

Un exemple de validation du devis

Le test QuoteRequestTypeTest utilise une demande fictive : douze polos fournis par le client avec un motif brodé. Le logo et le nom d’organisme peuvent rester absents. Les autres champs reçoivent des valeurs valides.

Extrait du dépôt • tests/Form/QuoteRequestTypeTest.php • lignes 13 à 19

```
13  private const VALID_DATA = [
14      'articleType' => 'Polo', 'articleOrigin' => 'client', 'markingType' => 'broderie',
15      'quantity' => 12, 'firstName' => 'Camille', 'lastName' => 'Exemple',
16      'organizationType' => 'particulier', 'organizationName' => '',
17      'email' => 'camille@example.test', 'phone' => '0600000000',
18      'message' => 'Douze polos avec un motif brodé.',
19  ];
```

Variation du jeu de données	Attendu	Obtenu
Données valides ci-dessus	Formulaire valide	Valide
Même demande avec quantity = 0	Formulaire invalide	Invalide
Même demande avec email = invalide	Formulaire invalide	Invalide

Ces résultats proviennent de la suite locale exécutée le 18 octobre 2026. Dans le test isolé du formulaire, le CSRF est désactivé pour cibler les contraintes de saisie. Les protections des parcours sont vérifiées séparément par les tests des contrôleurs.